

Designing a Distributed Flight Booking System for ACME Organization

Anis Abdellatif - 5621

31/01/2025

Abstract

This document presents the design of a distributed flight booking system (FBS) for the ACME Organization. The redesigned system addresses the challenges of scalability, performance, and fault tolerance by distributing responsibilities across multiple nodes. We identify both functional and non-functional requirements, present a system architecture with key components and interactions, and discuss how the design meets the specified requirements.

1 Introduction

The ACME Organization currently operates a flight booking system that is based on a single server. With increasing demands such as hundreds of flights sending in data and hundreds of thousands of concurrent traveler requests, the existing system falls short in meeting scalability, performance, and fault tolerance requirements. In this exercise, we redesign the system into a distributed architecture that not only handles a large number of concurrent operations but also ensures data consistency and high availability, thereby enhancing the overall customer experience.

The new system is designed to:

- Ingest periodic flight data from partner airlines.
- Provide real-time flight information to travelers.
- Process flight reservations while maintaining price guarantees.

2 Requirements

We begin by identifying and categorizing the requirements extracted from the problem statement.

2.1 Functional Requirements

FR1 – Flight Data Ingestion:

The system must receive updated flight data (routes, occupancies, and prices) from partner airlines every hour.

FR2 – Flight Search and Status Query:

Travelers must be able to search for flights matching their criteria and view real-time status information.

FR3 – Flight Reservation and Booking:

The system must allow travelers to book seats and ensure that the price promised to the customer remains valid for a brief period before finalization.

2.2 Non-Functional Requirements

NFR1 – Scalability:

The system must handle hundreds of flights sending in data and hundreds of thousands of concurrent traveler requests.

NFR2 – Performance (Low Latency):

Traveler requests (for flight search, status, and bookings) must be processed within a matter of seconds.

NFR3 – Fault Tolerance and High Availability:

The system must remain operational even in the event of individual node or service failures.

NFR4 – Data Consistency:

Once a price is promised, it must remain valid for a short period, ensuring consistency even during concurrent updates.

3 System Architecture

The following section outlines the distributed system architecture designed to address the identified requirements. The architecture is composed of several components that work together to provide scalability, performance, and fault tolerance.

3.1 Key Components

C1 API Gateway / Load Balancer: Acts as the entry point for both traveler requests and partner airline data feeds. It distributes requests evenly across the available services.

C2 Flight Data Ingestion Service: Receives periodic updates from partner airlines. It uses a messaging system (e.g., Kafka) to queue incoming data for asynchronous processing.

C3 Application Microservices:

- **Flight Query Service:** Handles search and status requests from travelers.
- **Booking Service:** Manages flight reservations, ensuring that the promised price remains consistent.

C4 Distributed Cache: Provides fast, in-memory access to flight information and pricing (using systems such as Redis or Memcached) to reduce latency.

C5 Distributed Data Store: Stores flight data, occupancy details, pricing, and booking records. This could be a NoSQL (e.g., Cassandra) or a NewSQL database with replication and partitioning.

C6 Coordination Service: Manages distributed locks and consensus (using tools like ZooKeeper or etcd) to ensure consistency during critical booking operations.

C7 Monitoring and Logging Service: Continuously monitors system health, performance, and logs anomalies to facilitate quick recovery from failures.

3.2 Architecture Diagram

Figure 1 illustrates the high-level system architecture.

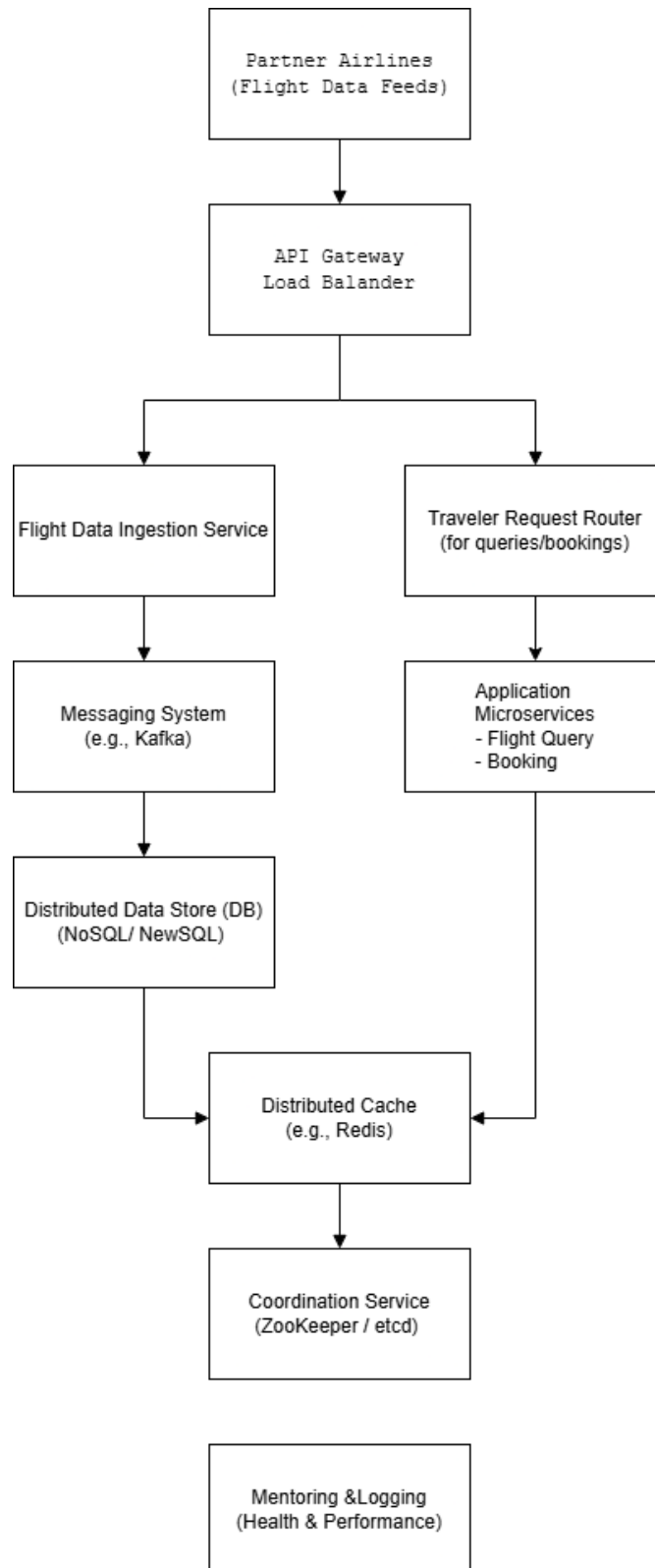


Figure 1: High-level Distributed Architecture for the Flight Booking System

4 Discussion: Addressing the Requirements

This section explains how the proposed architecture meets each of the identified requirements.

4.1 Functional Requirements

- **FR1 (Flight Data Ingestion):** The dedicated **Flight Data Ingestion Service**, paired with a robust messaging system, ensures that periodic flight data from partner airlines is reliably received and processed asynchronously. This decoupling smooths out spikes in data load.
- **FR2 (Flight Search and Status Query):** The **Flight Query Service** processes traveler queries rapidly by utilizing a **Distributed Cache** for fast access to frequently requested flight information, thus ensuring real-time responses.
- **FR3 (Flight Reservation and Booking):** The **Booking Service** handles seat reservations and enforces the promised price guarantee. It leverages the **Coordination Service** to manage distributed locks and maintain consistency across booking transactions.

4.2 Non-Functional Requirements

- **NFR1 (Scalability):** The use of an **API Gateway/Load Balancer**, horizontally scalable microservices, and a distributed data store allows the system to efficiently scale out as the number of flights and traveler requests increases.
- **NFR2 (Performance – Low Latency):** A **Distributed Cache** ensures that frequently accessed data is readily available, while the separation of concerns among microservices enables parallel processing, thus minimizing response times.
- **NFR3 (Fault Tolerance and High Availability):** Redundancy is built into each layer of the architecture. The distributed data store replicates data, and multiple instances of microservices prevent a single point of failure, ensuring the system remains operational even when individual components fail.
- **NFR4 (Data Consistency):** The **Coordination Service** manages distributed locks and consensus for critical operations (such as booking transactions), ensuring that once a price is promised, it remains valid across the distributed system despite concurrent updates.

5 Conclusion

In this document, we have outlined the design of a distributed flight booking system that addresses the scalability, performance, and fault tolerance challenges faced by the ACME Organization. By identifying both functional and non-functional requirements and mapping them to a modular, multi-component architecture, the system is well-equipped to handle the high volume of data and concurrent traveler requests while ensuring consistency and reliability. This design represents a modern approach to building resilient distributed systems capable of evolving alongside growing business demands.

Evaluation of the Peer-to-Peer Network Implementation for File Sharing

Anis Abdellatif - 5621

31/01/2025

Introduction

This project extends a custom communication protocol to create a hybrid peer-to-peer (P2P) network architecture for file sharing, combining centralized coordination with decentralized resource distribution. The system consists of a Peer Management Server (PMS) that tracks peer metadata and availability, and a network of peers capable of direct file transfers. By reusing modular components (e.g., TCP/UDP clients) from prior work, the prototype demonstrates practical distributed systems (DS) principles while prioritizing rapid development. While the core functionality—peer registration, heartbeat monitoring, and file transfers—operates effectively, the implementation intentionally prioritizes simplicity over robustness to align with the scope of a proof-of-concept.

This project was inspired by the Exo project ([GitHub Link](#)), which is a decentralized P2P network designed for training AI agents on different devices.

Alignment with Distributed Systems Challenges

This project addresses three foundational DS challenges:

Decentralized Resource Sharing

By enabling peers to share files directly, the system avoids reliance on centralized storage, distributing the network's load and reducing bottlenecks. This mirrors real-world P2P systems like BitTorrent, where scalability depends on decentralization.

Dynamic Membership Management

The PMS uses heartbeat messages to detect offline peers, addressing the inherent volatility of distributed networks. However, unlike fully decentralized systems (e.g., blockchain networks using gossip protocols), this approach centralizes membership tracking, creating a trade-off between simplicity and fault tolerance.

Hybrid Architectures

The PMS acts as a coordinator, while peers handle resource distribution—a model seen in systems like Napster. This hybrid design illustrates how centralized components can simplify discovery in decentralized networks without fully sacrificing scalability.

Strengths of the Implementation

- **Effective Hybrid Architecture:** The PMS's centralized coordination simplifies peer discovery and metadata management, while direct peer-to-peer transfers reduce server load. This separation of concerns aligns with the *coordinator-worker* pattern common in distributed systems.
- **Fault Detection Mechanism:** Periodic heartbeat messages provide basic fault detection, ensuring the PMS maintains an accurate view of active peers. This is analogous to keep-alive mechanisms in distributed consensus algorithms (e.g., ZooKeeper's session management).

- **Code Reusability:** Modular components like the TCP/UDP clients promote maintainability and reduce redundancy. For example, encapsulating network communication logic in reusable classes follows the facade pattern, abstracting complexity for higher-level components.
- **Baseline Scalability:** The P2P model allows horizontal scaling: adding peers increases the network's aggregate bandwidth and storage capacity, a key advantage over client-server architectures.

Weaknesses and Limitations

- **Single-Source File Transfers:** Unlike modern P2P systems (e.g., BitTorrent), files are downloaded from a single peer rather than split into chunks from multiple sources. This limits parallelism and resilience, violating the *divide-and-conquer* principle central to distributed computing.
- **Centralized Single Point of Failure (SPOF):** The PMS's in-memory storage and lack of redundancy make it a SPOF. If it crashes, the network loses all peer metadata—a critical flaw compared to distributed coordination services like etcd or Consul, which use consensus algorithms for fault tolerance.
- **Minimal Error Handling:** File transfers lack checksums or retry logic, risking data corruption and incomplete transfers. In distributed systems, idempotent operations and acknowledgment protocols (e.g., TCP's ACK mechanism) are essential for reliability.
- **Security Oversight:** The absence of encryption or authentication exposes the system to eavesdropping and spoofing attacks. Real-world DS often employ TLS for communication and OAuth2 for identity management, which this prototype omits.
- **Ephemeral Data Storage:** Storing peer metadata in memory limits persistence and scalability. Distributed databases (e.g., Apache Cassandra) or distributed hash tables (DHTs) would improve resilience and support larger networks.
- **Technical Debt:** Rapid development led to procedural code with limited error handling, violating SOLID principles. For example, the PeerManager class assumes uninterrupted network connectivity, risking silent failures during edge cases.

Areas for Improvement

- **Chunk-Based File Transfers:** Implement rarest-first chunk selection (as in BitTorrent) to allow parallel downloads from multiple peers. This would improve speed and redundancy, leveraging the P2P network's distributed nature more effectively.
- **Decentralize the PMS:** Replace the centralized PMS with a DHT-based system where peers self-organize and share metadata. Alternatively, adopt a federated server model (e.g., Mastodon) to distribute coordination across multiple PMS instances.
- **Data Integrity Measures:** Add SHA-256 checksums to file transfers and implement automatic retries for failed chunks. For atomicity, adopt a two-phase commit protocol to ensure transfers complete fully or roll back.
- **Security Enhancements:**
 - Encrypt PMS-peer communication using TLS.
 - Use HMAC signatures to validate heartbeat messages.
 - Introduce role-based access control (RBAC) to restrict file access.
- **Persistent Storage:** Migrate PMS metadata to a distributed database (e.g., Redis Cluster) with replication. This would ensure data survives server restarts and scales to larger networks.
- **Load Testing and Metrics:** Define quantitative metrics (e.g., time to join/leave the network, transfer throughput under load) to evaluate performance. Tools like Locust could simulate traffic and identify bottlenecks.

Conclusion

This project successfully demonstrates core distributed systems concepts through a functional P2P file-sharing network. Its hybrid architecture balances simplicity with scalability, while the heartbeat mechanism addresses dynamic membership—a critical challenge in DS. However, the prototype’s limitations in fault tolerance, security, and data integrity highlight the trade-offs inherent in prioritizing rapid development over production-grade robustness.

The implementation aligns with the course’s learning objectives by providing hands-on experience in protocol design and evaluation. Future work could transform this prototype into a resilient, decentralized system by integrating chunk-based transfers, consensus protocols, and security frameworks. Ultimately, the project underscores a key lesson in distributed systems: engineering a functional prototype is only the first step; the true challenge lies in anticipating and mitigating real-world failures.